TEMPLE COLLEGE
WORKFORCE DEVELOPMENT

CHIEFOS

STUDENT REVIEW PACKET

Week 3

Making things, reading files, sudo & apt, permissions — Escape Night

Phase 1 — Foundations · Weeks 1–6 · Modules 00–02

No D2L login required. This is the same lesson text posted in Brightspace for this week. Work the lessons in order on your own machine. Keep each checkpoint screenshot or pasted output. Bring it to the live session, or email it to your instructor, instead of submitting in D2L. Module quizzes stay in D2L and are not included here.

In this packet

1. 01.02 Making Things
2. 01.03 Looking Inside Files
3. 01.04 Sudo and System Updates
4. 01.05 Installing Software
5. 01.06 Permissions Drills
6. Live lab — Terminal Escape Night

Learning objective: Create the ~/docker-course working directory and practice creating, copying, moving, renaming, and deleting files and directories with `mkdir`, `touch`, `cp`, `mv`, and `rm`.

Lesson 02: Making Things

By the end, you will have created the ~/docker-course/ working directory you'll use for the rest of the module.

Estimated time: ~15 minutes of typing

---

1. Start from home.

```
cd ~
```

2. Confirm you are in your home directory.

```
pwd
```

```
/home/your-username
```

3. Create the working directory for this course.

```
mkdir docker-course
```

4. Confirm it exists.

```
ls
```

(You should see `docker-course` in the list.)

5. Move into it.

```
cd docker-course
```

6. Confirm you are inside the new directory.

```
pwd
```

```
/home/your-username/docker-course
```

7. Create a subdirectory for practice files.

```
mkdir practice
```

8. Create another subdirectory.

```
mkdir notes
```

9. List to see both.

```
ls
```

```
notes  practice
```

10. Try creating a nested directory path in one command.

```
mkdir -p projects/web/app
```

11. Confirm the full path was created.

```
ls -la projects/web/
```

```
total 12
drwxrwxr-x 3 your-user your-user 4096 ... .
drwxrwxr-x 3 your-user your-user 4096 ... ..
drwxrwxr-x 2 your-user your-user 4096 ... app
```

12. See the whole tree structure (using ls -R since tree may not be installed yet).

```
ls -R
```

```
.:
notes  practice  projects

./notes:

./practice:

./projects:
web

./projects/web:
app

./projects/web/app:
```

13. Create an empty file using touch.

```
touch practice/file1.txt
```

14. Confirm it exists.

```
ls practice/
```

```
file1.txt
```

15. Create a few more files.

```
touch practice/file2.txt
```

```
touch practice/file3.txt
```

16. List them with details.

```
ls -la practice/
```

(You should see file1.txt, file2.txt, and file3.txt — all with 0 bytes.)

17. Copy a file.

```
cp practice/file1.txt practice/file1-backup.txt
```

18. Confirm the copy was created.

```
ls practice/
```

```
file1-backup.txt  file1.txt  file2.txt  file3.txt
```

19. Copy a file into a different directory.

```
cp practice/file1.txt notes/
```

20. Confirm the file landed in notes.

```
ls notes/
```

```
file1.txt
```

21. Rename a file using mv.

```
mv practice/file3.txt practice/renamed.txt
```

22. Confirm — file3.txt is gone, renamed.txt is there.

```
ls practice/
```

```
file1-backup.txt  file1.txt  file2.txt  renamed.txt
```

23. Move a file to a different directory.

```
mv practice/renamed.txt notes/
```

24. Confirm the file moved from practice to notes.

```
ls notes/
```

```
file1.txt  renamed.txt
```

```
ls practice/
```

```
file1-backup.txt  file1.txt  file2.txt
```

25. Remove a single file.

```
rm practice/file1-backup.txt
```

26. Confirm it's gone.

```
ls practice/
```

```
file1.txt  file2.txt
```

> :warning: **Don't do this:** `rm -rf /` or `rm -rf ~` — this would delete your entire system or home directory. Always double-check the path before running rm.

27. Create a temporary directory to practice deleting.

```
mkdir practice/temp-dir
```

28. Put a file in it.

```
touch practice/temp-dir/throwaway.txt
```

29. Try removing the directory with plain rm — this will fail.

```
rm practice/temp-dir
```

```
rm: cannot remove 'practice/temp-dir': Is a directory
```

30. Remove the directory and its contents with rm -r.

```
rm -r practice/temp-dir
```

> :warning: **Don't do this:** Never run `rm -r` on a directory without first checking what's inside it with `ls`. One wrong path and you lose files permanently.

31. Confirm it's gone.

```
ls practice/
```

```
file1.txt  file2.txt
```

32. Run mkdir -p on a path that already exists — it won't error.

```
mkdir -p practice
```

(No output, no error. That's why -p is safe to use.)

33. Go home and confirm your docker-course structure.

```
cd ~
```

```
ls -R docker-course/
```

```
docker-course/:
notes  practice  projects

docker-course/notes:
file1.txt  renamed.txt

docker-course/practice:
file1.txt  file2.txt

docker-course/projects:
web

docker-course/projects/web:
app

docker-course/projects/web/app:
```

Checkpoint

```
ls ~/docker-course/
```

You should see: `notes practice projects`

```
ls ~/docker-course/practice/
```

You should see: `file1.txt file2.txt`

```
ls ~/docker-course/notes/
```

You should see: `file1.txt renamed.txt`

```
ls ~/docker-course/projects/web/app/
```

Should show an empty directory (no error).

Lab

Objective: Build the `~/docker-course/` working directory and manage files with `mkdir`, `touch`, `cp`, `mv`, and `rm`.

Task: Work through the 33 drills in this lesson — the directory structure they leave behind is the artifact. Then run the four Checkpoint commands in one terminal.

Submit: One screenshot (or pasted transcript) of the four Checkpoint commands and their output: `ls ~/docker-course/`, `ls ~/docker-course/practice/`, `ls ~/docker-course/notes/`, `ls ~/docker-course/projects/web/app/`.

Pass criteria:

- [] `ls ~/docker-course/` shows `notes practice projects`.
- [] `ls ~/docker-course/practice/` shows `file1.txt file2.txt`.
- [] `ls ~/docker-course/notes/` shows `file1.txt renamed.txt`.
- [] `ls ~/docker-course/projects/web/app/` returns with no error (empty directory).

Unlocks: 03-looking-inside-files.md

Next up: 03-looking-inside-files.md — Read and edit files

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Create, read, and edit files from the terminal using `echo` with `>` and `>>` redirection, `cat`, `head`, `tail`, `less`, `wc -l`, and `nano`.

Lesson 03: Looking Inside Files

By the end, you will have created, read, and edited files using the terminal.

~15 minutes of typing

---

1. Go to your course directory.

```
cd ~/docker-course
```

2. Confirm.

```
pwd
```

```
/home/your-username/docker-course
```

3. Create a notes file by writing text into it with echo.

```
echo "This is my first note" > notes/notes.txt
```

(The `>` redirects the output of echo into a file, creating it if it doesn't exist.)

4. Read the file with cat.

```
cat notes/notes.txt
```

```
This is my first note
```

5. Add a second line using >> (append, don't overwrite).

```
echo "This is my second note" >> notes/notes.txt
```

6. Read it again to see both lines.

```
cat notes/notes.txt
```

```
This is my first note
This is my second note
```

7. Add a few more lines so we have enough to practice with.

```
echo "Line 3: learning cat" >> notes/notes.txt
```

```
echo "Line 4: learning head" >> notes/notes.txt
```

```
echo "Line 5: learning tail" >> notes/notes.txt
```

```
echo "Line 6: learning less" >> notes/notes.txt
```

```
echo "Line 7: almost done" >> notes/notes.txt
```

```
echo "Line 8: last line" >> notes/notes.txt
```

8. Read the full file again.

```
cat notes/notes.txt
```

```
This is my first note
This is my second note
Line 3: learning cat
Line 4: learning head
Line 5: learning tail
Line 6: learning less
Line 7: almost done
Line 8: last line
```

9. Show only the first 3 lines with head.

```
head -n 3 notes/notes.txt
```

```
This is my first note
This is my second note
Line 3: learning cat
```

10. Show only the last 3 lines with tail.

```
tail -n 3 notes/notes.txt
```

```
Line 6: learning less
Line 7: almost done
Line 8: last line
```

11. Now look at a longer system file with less -- this opens a scrollable viewer.

```
less /etc/passwd
```

(Use arrow keys to scroll. **Press q to quit.** This is critical -- you must press q to exit less.)

12. Use head on a system file to peek at the first 5 lines.

```
head -n 5 /etc/passwd
```

```
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
sys:x:3:3:sys:/dev:/usr/sbin/nologin
sync:x:4:65534:sync:/bin:/bin/sync
```

13. Use tail on a system file to see the last 5 lines.

```
tail -n 5 /etc/passwd
```

(Output will vary -- you'll see the last 5 user accounts on the system.)

14. Now try tail -f -- this watches a file for new content in real time.

```
tail -f notes/notes.txt
```

(This will show the last 10 lines and then WAIT. It looks frozen -- it's not. It's waiting for new data.)

Press Ctrl+C to stop tail -f. This is how you exit it.

15. Now edit a file with nano -- a simple terminal text editor.

```
nano notes/notes.txt
```

(nano opens the file for editing.)

Inside nano:

- Type a new line at the bottom: `Line 9: edited with nano`
- Press **Ctrl+O** to save (then press Enter to confirm the filename)
- Press **Ctrl+X** to exit nano

16. Confirm your edit was saved.

```
cat notes/notes.txt
```

```
This is my first note
This is my second note
Line 3: learning cat
Line 4: learning head
Line 5: learning tail
Line 6: learning less
Line 7: almost done
Line 8: last line
Line 9: edited with nano
```

17. Use cat to look at a system file -- /etc/hostname.

```
cat /etc/hostname
```

(Output will show your machine's hostname.)

18. Count how many lines are in your notes file using wc.

```
wc -l notes/notes.txt
```

```
9 notes/notes.txt
```

> **Don't do this:** `echo "text" > important-file.txt` when you mean `>>`. A single `>` OVERWRITES the file. Double `>>` APPENDS. This mistake has destroyed many files.

Checkpoint

```
cat ~/docker-course/notes/notes.txt
```

You should see 9 lines, ending with "Line 9: edited with nano".

```
wc -l ~/docker-course/notes/notes.txt
```

You should see `9`.

```
head -n 1 ~/docker-course/notes/notes.txt
```

You should see: `This is my first note`

```
tail -n 1 ~/docker-course/notes/notes.txt
```

You should see: `Line 9: edited with nano`

Lab

Objective: Create, read, and edit files from the terminal with `echo`, `cat`, `head`, `tail`, `less`, and `nano`.

Task: Work through the drills to build the nine-line `notes.txt`, including the final line added and saved inside nano. Then run the four Checkpoint commands in one terminal.

Submit: One screenshot (or pasted transcript) of the four Checkpoint commands and their output: `cat ~/docker-course/notes/notes.txt`, `wc -l ~/docker-course/notes/notes.txt`, `head -n 1 ~/docker-course/notes/notes.txt`, `tail -n 1 ~/docker-course/notes/notes.txt`.

Pass criteria:

- [ ] `cat` shows all 9 lines of the file.
- [ ] `wc -l` reports `9`.
- [ ] `head -n 1` prints `This is my first note`.
- [ ] `tail -n 1` prints `Line 9: edited with nano` — proof you saved from nano.

Unlocks: 04-sudo-and-updates.md

Next up: 04-sudo-and-updates.md -- System updates with sudo

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Perform a full system update with `sudo apt update`, `sudo apt upgrade -y`, and `sudo apt autoremove -y`.

Lesson 04: Sudo and System Updates

By the end, you will have run a full system update on your Ubuntu machine.

~10 minutes of typing (plus wait time for downloads)

---

1. Start from home.

```
cd ~
```

2. Confirm.

```
pwd
```

```
/home/your-username
```

3. Confirm sudo works — you'll need it for every command in this lesson.

```
sudo whoami
```

```
root
```

4. Update the package list — this downloads the latest list of available software.

```
sudo apt update
```

```
Hit:1 http://archive.ubuntu.com/ubuntu jammy InRelease
Get:2 http://archive.ubuntu.com/ubuntu jammy-updates InRelease [119 kB]
...
Reading package lists... Done
Building dependency tree... Done
X packages can be upgraded. Run 'apt list --upgradable' to see them.
```

(Output will vary. The last line tells you how many packages can be upgraded.)

5. See which packages have updates available.

```
sudo apt list --upgradable
```

```
Listing... Done
package1/jammy-updates 1.2.3 amd64 [upgradable from: 1.2.2]
...
```

(You may see many packages or none. Both are fine.)

6. Upgrade all packages — the -y flag auto-confirms.

```
sudo apt upgrade -y
```

```
Reading package lists... Done
Building dependency tree... Done
Calculating upgrade... Done
The following packages will be upgraded:
  ...
X upgraded, 0 newly installed, 0 to remove and 0 not upgraded.
...
```

(This may take a few minutes. Wait for it to finish.)

7. Run update again to confirm everything is current.

```
sudo apt update
```

```
...
All packages are up to date.
```

(You should see "All packages are up to date" or "0 packages can be upgraded.")

8. Clean up any packages that are no longer needed.

```
sudo apt autoremove -y
```

```
Reading package lists... Done
Building dependency tree... Done
0 to remove.
```

(Output will vary. This removes orphaned dependencies.)

9. Check that your course directory is still intact — updates don't touch your files, but it's a good habit to verify.

```
ls ~/docker-course/
```

```
notes  practice  projects
```

10. Check your notes file is still there.

```
cat ~/docker-course/notes/notes.txt
```

(You should see all 9 lines from the previous lesson.)

> **Don't do this:** `sudo apt upgrade` without the `-y` flag during a scripted install — it will pause and wait for confirmation, which can hang an automated process. For interactive use (like right now), either way is fine.

Checkpoint

```
sudo apt update
```

You should see "All packages are up to date" or "0 packages can be upgraded."

```
sudo whoami
```

You should see `root`.

```
pwd
```

You should be in `/home/your-username`.

Lab

Objective: Run a full system update with `sudo apt update` and `sudo apt upgrade`, leaving the system with nothing left to upgrade.

Task: Work through the update drills — refresh the package list, upgrade everything, autoremove — then run the Checkpoint commands.

Submit: One screenshot (or pasted transcript) showing the tail end of a final `sudo apt update` run plus the output of `sudo whoami`.

Pass criteria: (verify the final state, not exact text — package counts and mirror lines differ on every machine)

- [ ] `sudo apt update` ends with `All packages are up to date.` (or reports `0 packages can be upgraded`).
- [ ] `sudo whoami` prints `root`.
- [ ] No error lines in the output. (A `dpkg lock` error means another apt process was running — wait for it and re-run.)

Unlocks: 05-installing-software.md

Next up: 05-installing-software.md — Install the packages we need

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Install and verify the Docker prerequisite packages (curl, wget, git, htop, tree, ca-certificates, gnupg) using `sudo apt install`.

Lesson 05: Installing Software

By the end, you will have installed all the prerequisite packages needed for Docker (in Module 02).

~10 minutes of typing

---

1. Start from home.

```
cd ~
```

2. Confirm.

```
pwd
```

```
/home/your-username
```

3. Always update the package list before installing anything.

```
sudo apt update
```

(Wait for it to finish.)

4. Install curl — a tool for making HTTP requests from the terminal.

```
sudo apt install -y curl
```

```
Reading package lists... Done
...
curl is already the newest version (x.x.x).
```

(If already installed, apt will tell you. That's fine — type the command anyway.)

5. Verify curl is installed.

```
curl --version
```

```
curl 7.81.0 (x86_64-pc-linux-gnu) ...
```

(Version number will vary.)

6. Install wget — another download tool, used by many install scripts.

```
sudo apt install -y wget
```

7. Verify wget.

```
wget --version | head -n 1
```

```
GNU Wget 1.21.2 built on linux-gnu.
```

8. Install git — version control, needed for many Docker workflows.

```
sudo apt install -y git
```

9. Verify git.

```
git --version
```

```
git version 2.34.1
```

10. Install htop — a better process viewer than top.

```
sudo apt install -y htop
```

11. Verify htop works (press q to quit immediately).

```
htop
```

(You'll see a colorful process viewer. **Press q to quit.**)

12. Install tree — shows directory structures visually.

```
sudo apt install -y tree
```

13. Try tree on your course directory.

```
tree ~/docker-course/
```

```
/home/your-username/docker-course/
├── notes
│   ├── file1.txt
│   ├── notes.txt
│   └── renamed.txt
├── practice
│   ├── file1.txt
│   └── file2.txt
└── projects
    └── web
        └── app
```

14. Install ca-certificates — needed for secure HTTPS connections (Docker install requires this).

```
sudo apt install -y ca-certificates
```

15. Install gnupg — handles encryption keys (Docker's GPG key needs this).

```
sudo apt install -y gnupg
```

16. Install them all in one line — this is how you'll often see it in tutorials. Type it out.

```
sudo apt install -y curl wget git htop tree ca-certificates gnupg
```

```
Reading package lists... Done
...
0 newly installed, 0 to remove...
```

(Everything should already be installed. This is practice for combining packages in one command.)

17. Record what you installed in your notes.

```
echo "Installed: curl, wget, git, htop, tree, ca-certificates, gnupg" >> ~/docker-course/notes/notes.txt
```

18. Verify your note was added.

```
tail -n 1 ~/docker-course/notes/notes.txt
```

```
Installed: curl, wget, git, htop, tree, ca-certificates, gnupg
```

Checkpoint

```
curl --version | head -n 1
```

You should see a curl version number.

```
git --version
```

You should see a git version number.

```
tree --version
```

You should see a tree version number.

```
which wget
```

You should see `/usr/bin/wget`.

```
dpkg -l | grep gnupg
```

You should see gnupg in the list.

Lab

Objective: Install the seven prerequisite packages (curl, wget, git, htop, tree, ca-certificates, gnupg) and verify each one answers.

Task: Install each package by working through the drills — type every command even if apt says it's already installed. Then run the five Checkpoint commands in one terminal.

Submit: One screenshot (or pasted transcript) of the five Checkpoint commands and their output: `curl --version | head -n 1`, `git --version`, `tree --version`, `which wget`, `dpkg -l | grep gnupg`.

Pass criteria: (version numbers vary by machine — any version string passes)

- [] `curl --version | head -n 1` prints a curl version line.
- [] `git --version` prints a git version line.
- [] `tree --version` prints a tree version line.
- [] `which wget` prints `/usr/bin/wget`.
- [] `dpkg -l | grep gnupg` shows gnupg in the installed list.

Unlocks: 06-permissions-drills.md

Next up: 06-permissions-drills.md — Understand and change file permissions

Lab — do it for real. Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Learning objective: Read permission strings and change file permissions and ownership using `chmod` in symbolic and numeric forms and `chown` with sudo.

Lesson 06: Permissions Drills

By the end, you will be able to read, set, and change file permissions and ownership.

~15 minutes of typing

---

1. Go to your practice directory.

```
cd ~/docker-course/practice
```

2. Confirm.

```
pwd
```

```
/home/your-username/docker-course/practice
```

3. Look at the current permissions of your files.

```
ls -la
```

```
total 8
drwxrwxr-x 2 your-user your-user 4096 ... .
drwxrwxr-x 5 your-user your-user 4096 ... ..
-rw-rw-r-- 1 your-user your-user    0 ... file1.txt
-rw-rw-r-- 1 your-user your-user    0 ... file2.txt
```

(Reading the permission string `-rw-rw-r--`:

- Position 1: `-` = file, `d` = directory
- Positions 2-4: owner permissions (rwx)
- Positions 5-7: group permissions (rwx)
- Positions 8-10: everyone else (rwx))

4. Create a simple shell script.

```
echo '#!/bin/bash' > hello.sh
```

5. Add a command to the script.

```
echo 'echo "Hello from my script!"' >> hello.sh
```

6. Check what's in the file.

```
cat hello.sh
```

```
#!/bin/bash
echo "Hello from my script!"
```

7. Look at its permissions.

```
ls -la hello.sh
```

```
-rw-rw-r-- 1 your-user your-user 42 ... hello.sh
```

(Notice: no `x` in the permissions -- the file is NOT executable yet.)

8. Try to run it -- this will fail.

```
./hello.sh
```

```
bash: ./hello.sh: Permission denied
```

9. Make it executable with chmod +x.

```
chmod +x hello.sh
```

10. Check the permissions again.

```
ls -la hello.sh
```

```
-rwxrwxr-x 1 your-user your-user 42 ... hello.sh
```

(Now you see `x` in the permissions.)

11. Run it.

```
./hello.sh
```

```
Hello from my script!
```

12. Now practice setting permissions with numbers -- set to 644 (owner read/write, everyone else read-only).

```
chmod 644 hello.sh
```

13. Check.

```
ls -la hello.sh
```

```
-rw-r--r-- 1 your-user your-user 42 ... hello.sh
```

(6 = rw-, 4 = r--, 4 = r--)

14. Try to run it -- it's no longer executable.

```
./hello.sh
```

```
bash: ./hello.sh: Permission denied
```

15. Set it to 755 (owner full, everyone else read+execute).

```
chmod 755 hello.sh
```

16. Check.

```
ls -la hello.sh
```

```
-rwxr-xr-x 1 your-user your-user 42 ... hello.sh
```

(7 = rwx, 5 = r-x, 5 = r-x)

17. Run it again to confirm.

```
./hello.sh
```

```
Hello from my script!
```

18. Create another file to practice chown.

```
touch owned-by-me.txt
```

19. Check the owner.

```
ls -la owned-by-me.txt
```

```
-rw-rw-r-- 1 your-user your-user 0 ... owned-by-me.txt
```

20. Change the owner to root (requires sudo).

```
sudo chown root:root owned-by-me.txt
```

21. Check ownership again.

```
ls -la owned-by-me.txt
```

```
-rw-rw-r-- 1 root root 0 ... owned-by-me.txt
```

(Now root owns this file.)

22. Change it back to yourself.

```
sudo chown $USER:$USER owned-by-me.txt
```

23. Confirm.

```
ls -la owned-by-me.txt
```

```
-rw-rw-r-- 1 your-user your-user 0 ... owned-by-me.txt
```

24. Look at permissions on a system file to practice reading.

```
ls -la /etc/passwd
```

```
-rw-r--r-- 1 root root 1847 ... /etc/passwd
```

(644 -- owner can read/write, everyone else can only read.)

25. Look at a directory's permissions.

```
ls -la /tmp
```

(Notice the `d` at the start -- it's a directory. Also notice the `t` at the end -- that's the sticky bit.)

26. Go home.

```
cd ~
```

> **Warning:** Don't do this: `chmod 777 /` or `chmod -R 777 ~` -- this gives everyone full access to everything. Never use 777 on a real server. It's a security disaster.

> **Warning:** Don't do this: `sudo chown -R root:root ~` -- this would make root own your entire home directory and lock you out of your own files.

Checkpoint

```
ls -la ~/docker-course/practice/hello.sh
```

You should see `-rwxr-xr-x` (permissions 755).

```
~/docker-course/practice/hello.sh
```

You should see: `Hello from my script!`

```
ls -la ~/docker-course/practice/owned-by-me.txt
```

Owner should be your username, not root.

```
stat -c "%a" ~/docker-course/practice/hello.sh
```

You should see: `755`

Lab

Objective: Read, set, and change file permissions and ownership with `chmod` and `chown`.

Task: Work through the drills — build `hello.sh`, make it executable, cycle it through 644 and 755, and practice `chown` both directions. Then run the four Checkpoint commands in one terminal.

Submit: One screenshot (or pasted transcript) of the four Checkpoint commands and their output: `ls -la ~/docker-course/practice/hello.sh`, running `~/docker-course/practice/hello.sh`, `ls -la ~/docker-course/practice/owned-by-me.txt`, `stat -c "%a" ~/docker-course/practice/hello.sh`.

Pass criteria:

- [ ] `ls -la` on `hello.sh` shows `-rwxr-xr-x`.
- [ ] Running the script prints `Hello from my script!`.
- [ ] `owned-by-me.txt` is owned by your username, not root.
- [ ] `stat -c "%a"` prints `755`.

Unlocks: 07-networking-drills.md

Next up: 07-networking-drills.md -- Test your network connectivity

> **Lab — do it for real.** Complete every step above on your own machine — type the commands, don't copy-paste. **Submit:** a screenshot or pasted output of the *Checkpoint* section (or this lesson's final verification steps) Keep it. If you cannot submit in D2L, bring it to class or email it to your instructor. Completing the lab unlocks the next lesson.

Live lab — Terminal Escape Night

Wednesday live lab · 6:00–8:00 PM · replaces the Week 3 clinic · reviews labs 01:02–06

Learning objective: Re-run navigation, `mkdir` / `cp` / `mv` / `rm -r`, `less` / `tail -f` /nano, and permissions (`ls -l` / `chmod` / `chown`) under a story — without new commands.

The story: a bad update has locked every door in the ChiefOS server building at 23:47 — and you're inside. Mouse dead. GUI dead. Terminal only. Fight through **four escape rooms**, each guarding one code word. Collect all four, write the escape plan, and get out. **Every room is solvable with commands you already used in labs 01:02–06.** The answer to every puzzle is written down inside the game. Read the room. It always tells you.

Do this before 6:00 PM — required

1. Download `setup.sh` from this topic (also attached to the D2L item).
2. In your WSL/Ubuntu terminal, run `bash setup.sh` — as yourself, **never** `sudo bash setup.sh`.
3. It asks for your password **once**, near the end. The prompt types invisibly — keep typing, press Enter.
4. Verify: `ls ~/escape-night` shows **four room folders** plus a README.
5. **Don't go in yet.** Exploring early is cheating yourself out of the race.

```
bash setup.sh
ls ~/escape-night
```

Mac note: if you hit `chown: root: illegal group name`, run `sed -i '' 's/chown root:root/chown root/' setup.sh` then `bash setup.sh --reset`.

How the race works

Piece	What it means
Teams	Team SUDO, Team NANO, Team CHMOD. Everyone types on their **own machine**. A team clears a room only when **every member** has that room's code word.
Roles (rotate every room)	**Driver** shares screen and types while narrating. **Reader** reads the room's files aloud. **Scribe** tracks code words and posts clears in main chat.
Rule #1	Nobody rides along. Finished early? Coach with **words only** — never touch a teammate's keyboard.
Hints	One hint token for the night (Zoom “Ask for Help”). Unlimited free hint: **read the room's files again**.

Tonight's clock

Clock	Block	What happens
6:00	Check-in	Post a ✓ in chat when `ls ~/escape-night` shows four rooms. Setup rescue for anyone stuck.
6:20	Launch	The story, team assignments, rules, breakouts.
6:30	The escape	50:00 on the clock.
7:20	Debrief	Each team teaches one room. Exit ticket: read `ls -l ~/escape-night/escape` out loud.
7:40	Close	Confirm labs 01:02–06 submitted. Week 4 phase-gate preview.

Train before tonight — seven escape skills

If you can do these, you can escape. Rusty on any? Re-run that lab this afternoon.

#	Can you…	Practice	From lab
1	Move around and look inside things without a mouse	`pwd` · `ls` · `cd` · `cd ..` · `cat`	01:01–02
2	Build, copy, rename, and carefully destroy	`mkdir` · `cp` · `mv` · `rm -r`	01:02
3	Page through a long file — and get back out	`less file` (space = next, q = quit)	01:03
4	Watch a file update live in a second terminal	`tail -f file` (Ctrl+C to stop)	01:03
5	Edit, save, and exit in nano — no panic	`nano file` · Ctrl+O, Enter · Ctrl+X	01:03
6	Use sudo when the system says you can't	`sudo whoami` · `sudo chown`	01:04, 01:06
7	Read an `ls -l` permission string and change it	`ls -l` · `chmod` · `chmod +x`	01:06

Skill 7 is the boss fight — and the exact skill next week's Module 01 phase-gate tests.

Rules of the building

- Don't enter `~/escape-night` before the race starts.
- Coach with words only. No web searches — everything is in the rooms.
- Notes, this page, and your lab files are legal.

- Everything happens inside `~/escape-night/`. The game contains exactly one `rm -r` — read the path twice, then read it again.
- Wrecked a room? `bash setup.sh --reset` rebuilds it in seconds.

Submit. Screenshot the victory banner in your terminal and post it in Zoom chat — that's tonight's participation artifact. Still owe labs 01:02–06? Tonight *is* the review. Finish and submit those in D2L by Sunday.